

Network Architecture

From VMware vSwitch to Linux Bridge, OVS & SR-IOV

Network configuration is the most critical aspect of VM migration. This guide maps VMware virtual networking concepts to Linux equivalents — bridges, OVS, VLANs, bonds, SR-IOV, and firewall rules. `hyper2kvm` auto-generates network configs from OVF metadata.

Linux Bridge | OVS | SR-IOV | VLAN Mapping | Bond Modes | nftables | MAC Preservation

Network Topology Mapping

VMware vSwitch/dvSwitch concepts map directly to Linux networking primitives.

VMware → Linux Network Mapping

VMware Concept	Linux Equivalent	Configuration	Notes
vSwitch	Linux bridge (br0)	ip link add br0 type bridge	Layer 2 switch, same functionality
dvSwitch	Open vSwitch (OVS)	ovs-vsctl add-br ovsbr0	Distributed features: flow tables, QoS, mirroring
Port Group	VLAN sub-interface	ip link add link eth0 name eth0.100 type vlan id 100	Each port group becomes a VLAN interface on bridge
vmnic (uplink)	Physical NIC (eth0/ens192)	ip link set eth0 master br0	Physical NIC added as bridge port
NIC Teaming	Linux bonding	ip link add bond0 type bond mode 802.3ad	LACP, active-backup, balance-xor
vMotion Network	Dedicated bridge + VLAN	Separate bridge on dedicated NIC	Used for live migration traffic
VMkernel Port	Host IP on bridge	ip addr add 10.0.0.1/24 dev br0	Management, storage, vMotion IPs
NSX Micro-segmentation	nftables + OVS flows	Per-VM firewall rules or OVS ACLs	libvirt nwfilter for simple cases

Auto-Detection from OVF/OVA Metadata

```
# h2kvmctl reads network config from OVF and generates libvirt XML
h2kvmctl convert --source vm-export.ova --emit-domain-xml

# OVF NetworkSection extracted:
#   Network "VM Network" → bridge=br0 (default)
#   Network "VLAN100-Production" → bridge=br-vlan100
#   Network "Storage" → bridge=br-storage

# Generated libvirt XML network interfaces:
# <interface type='bridge'>
#   <source bridge='br0' />
#   <model type='virtio' />
#   <mac address='52:54:00:ab:cd:01' />  <!-- preserved from source -->
# </interface>
```

Linux Bridge Configuration

The simplest and most reliable network backend for KVM virtual machines.

Bridge Setup Methods

NetworkManager (RHEL 9 / Rocky 9)

```
# Create bridge
nmcli con add type bridge \
  con-name br0 ifname br0 \
  ipv4.addresses 10.0.1.1/24 \
  ipv4.gateway 10.0.1.254 \
  ipv4.dns 10.0.1.10 \
  ipv4.method manual

# Add physical NIC as port
nmcli con add type bridge-slave \
  con-name br0-port1 \
  ifname ens192 master br0

# VLAN bridge
nmcli con add type bridge \
  con-name br-vlan100 ifname br100
nmcli con add type vlan \
  con-name ens192.100 \
  ifname ens192.100 dev ens192 \
  id 100 master br100
```

systemd-networkd (Ubuntu / Minimal)

```
# /etc/systemd/network/10-br0.netdev
[NetDev]
Name=br0
Kind=bridge

# /etc/systemd/network/20-br0.network
[Match]
Name=br0
[Network]
Address=10.0.1.1/24
Gateway=10.0.1.254
DNS=10.0.1.10

# /etc/systemd/network/30-ens192.network
[Match]
Name=ens192
[Network]
Bridge=br0

# Apply
systemctl restart systemd-networkd
```

Libvirt Network XML

```
<!-- Use existing bridge -->
<network>
  <name>production</name>
  <forward mode='bridge' />
  <bridge name='br0' />
</network>

<!-- NAT network (dev/test) -->
<network>
  <name>isolated</name>
```

Bridge Performance Tuning

- **STP:** Disable for VM-only bridges: `ip link set br0 type bridge stp_state 0`
- **Forward delay:** `ip link set br0 type bridge forward_delay 0`
- **Multicast snooping:** Disable if not needed: `echo 0 > /sys/class/net/br0/bridge/multicast_snooping`
- **TX queue:** `ip link set br0 txqueuelen 10000`
- **GRO/GSO:** `ethtool -K br0 gro on gso on`
- **Performance:** ~8 Gbps on 10GbE with tuning

```
<forward mode='nat' />
<bridge name='virbr1' />
<ip address='192.168.122.1'
    netmask='255.255.255.0'>
  <dhcp>
    <range start='192.168.122.100'
      end='192.168.122.200' />
  </dhcp>
</ip>
</network>
```

- **Latency:** ~20us additional vs direct passthrough
- Recommended for: most production workloads

Open vSwitch for Production

OVS provides advanced networking features matching VMware dvSwitch capabilities.

OVS vs Linux Bridge Decision Matrix

Feature	Linux Bridge	OVS	OVS-DPDK	When to Choose
Setup complexity	Simple	Medium	Complex	Bridge for <20 VMs, OVS for enterprise
VLAN support	Yes	Yes	Yes	Both handle VLANs; OVS has trunk/access port modes
Flow tables / OpenFlow	No	Yes	Yes	OVS for SDN, complex routing, ACLs
Port mirroring / SPAN	No	Yes	Yes	OVS for IDS/IPS traffic capture
QoS / rate limiting	Basic (tc)	Built-in	Built-in	OVS for per-VM bandwidth guarantees
LACP bond	Yes	Yes	Yes	Both support LACP; OVS adds load-balancing modes
Performance (10GbE)	8 Gbps	9 Gbps	Line rate	DPDK for telco/NFV workloads
Packets per second	2 Mpps	3 Mpps	14 Mpps	DPDK for high-PPS workloads

OVS Configuration

```
# Install OVS
dnf install openvswitch3.1 && systemctl enable --now openvswitch

# Create OVS bridge with VLAN trunking
ovs-vsctl add-br ovsbr0
ovs-vsctl add-port ovsbr0 ens192 trunk=100,200,300
ovs-vsctl add-port ovsbr0 vnet0 tag=100      # VM on VLAN 100
ovs-vsctl add-port ovsbr0 vnet1 tag=200      # VM on VLAN 200

# Port mirroring (send all traffic to IDS VM)
ovs-vsctl -- set Bridge ovsbr0 mirrors=@m \
  -- --id=@vnet0 get Port vnet0 \
  -- --id=@vnet-ids get Port vnet-ids \
  -- --id=@m create Mirror name=ids-mirror select-all=true output-port=@vnet-ids

# QoS: limit VM to 1 Gbps
ovs-vsctl set Interface vnet0 ingress_policing_rate=1000000
```

```
ovs-vsctl set Interface vnet0 ingress_policing_burst=100000
```

```
# Libvirt OVS integration
# <interface type='bridge'>
#   <source bridge='ovsbr0' />
#   <virtualport type='openvswitch' />
#   <vlan><tag id='100' /></vlan>
#   <model type='virtio' />
# </interface>
```

SR-IOV & Hardware Passthrough

Line-rate network performance by bypassing the hypervisor entirely.

Network Backend Performance Comparison

Backend	Throughput (10GbE)	Latency	PPS (64-byte)	CPU Overhead	Live Migration	Best For
virtio-net	9.4 Gbps	~50us	3.5 Mpps	Medium	Yes	General workloads
vhost-net	9.8 Gbps	~30us	5 Mpps	Low	Yes	Production (default)
SR-IOV VF	10 Gbps (line)	~5us	14.8 Mpps	Zero	No*	NFV, databases, latency-sensitive
PCI Passthrough	10 Gbps (line)	~5us	14.8 Mpps	Zero	No	Dedicated NIC per VM
macvtap (bridge)	9.2 Gbps	~25us	4 Mpps	Low	Yes	Simple, no bridge config
OVS-DPDK	10 Gbps (line)	~10us	14 Mpps	Dedicated cores	Yes	Telco NFV, virtual routers

* SR-IOV live migration possible with NVIDIA ConnectX-6+ bond failover mode

SR-IOV Setup

Enable SR-IOV Virtual Functions

```
# Enable IOMMU in kernel
# /etc/default/grub:
# GRUB_CMDLINE_LINUX="intel_iommu=on iommu=pt"
grub2-mkconfig -o /boot/grub2/grub.cfg

# Create 8 VFs on Intel X710
echo 8 > /sys/class/net/ens3f0/device/sriov_numvfs

# Persistent via udev rule
cat > /etc/udev/rules.d/70-sriov.rules <<'EOF'
ACTION=="add", SUBSYSTEM=="net", ENV{ID_NET_DRIVER}=="i40e", \
    ATTR{device/sriov_numvfs}="8"
EOF

# Verify VFs created
```

Libvirt SR-IOV XML

```
<!-- Assign VF to VM -->
<interface type='hostdev' managed='yes'>
  <source>
    <address type='pci'
      domain='0x0000' bus='0x03'
      slot='0x10' function='0x0' />
    </source>
    <mac address='52:54:00:ab:cd:ef' />
    <vlan><tag id='100' /></vlan>
  </interface>

<!-- Check IOMMU groups -->
# find /sys/kernel/iommu_groups/ \
#   -type l | sort -V
# Ensure VF is in its own group
```

```
ip link show ens3f0
# 8 VFs listed as ens3f0v0..ens3f0v7
```


Bond Modes & NIC Teaming Migration

VMware NIC teaming policies map to Linux bond modes for failover and load balancing.

VMware NIC Teaming → Linux Bond Mode Mapping

VMware Policy	Linux Bond Mode	Mode #	How It Works	Switch Requirement
Active/Standby	active-backup	mode=1	One active NIC, failover on link down	None (any switch)
IP Hash	balance-xor	mode=2	XOR of src/dst IP selects NIC	Static EtherChannel
LACP (dvSwitch)	802.3ad	mode=4	LACP negotiation, dynamic aggregation	LACP-capable switch
Source Port ID	balance-rr	mode=0	Round-robin across NICs	Static EtherChannel
Source MAC	balance-xor (layer2)	mode=2	XOR hash of MAC address	Static EtherChannel
Load Based (dvSwitch)	balance-alb	mode=6	Adaptive TX/RX load balancing	None (any switch)

Bond Configuration

```
# LACP bond with NetworkManager (most common enterprise config)
nmcli con add type bond con-name bond0 ifname bond0 \
    bond.options "mode=802.3ad,miimon=100,lacp_rate=fast,xmit_hash_policy=layer3+4"

# Add slave interfaces
nmcli con add type ethernet con-name bond0-port1 ifname ens192 master bond0
nmcli con add type ethernet con-name bond0-port2 ifname ens224 master bond0

# Create bridge on top of bond (for VMs)
nmcli con add type bridge con-name br0 ifname br0 \
    ipv4.addresses 10.0.1.1/24 ipv4.method manual
nmcli con add type bridge-slave con-name br0-bond0 ifname bond0 master br0

# Verify LACP negotiation
cat /proc/net/bonding/bond0
# Partner Mac Address: 00:1c:73:xx:xx:xx (switch side)
# Aggregator ID: 1 (both ports in same aggregation)
```

Migration Tip: Bond Mode Selection

Active-backup (mode=1) is the safest migration choice — works with any switch, no switch-side config needed. Start with mode=1, then upgrade to LACP (mode=4) after validating switch compatibility. h2kvmctl auto-detects VMware NIC teaming policy from OVF and suggests the matching Linux bond mode in the migration report.

Firewall & Security Group Migration

Mapping VMware NSX distributed firewall rules to Linux nftables and libvirt nwfilter.

NSX → nftables Rule Translation

VMware NSX Rules (Source)

```
# NSX Distributed Firewall
Rule 1: Web-Tier → App-Tier
  Source: web-sg (10.0.1.0/24)
  Dest:   app-sg (10.0.2.0/24)
  Port:   TCP 8080, 8443
  Action: Allow

Rule 2: App-Tier → DB-Tier
  Source: app-sg (10.0.2.0/24)
  Dest:   db-sg (10.0.3.0/24)
  Port:   TCP 5432 (PostgreSQL)
  Action: Allow

Rule 3: Default
  Source: Any
  Dest:   Any
  Action: Drop
```

Linux nftables (Target)

```
#!/usr/sbin/nft -f
table inet vm-firewall {
  chain forward {
    type filter hook forward
    priority 0; policy drop;

    # Web → App (TCP 8080, 8443)
    ip saddr 10.0.1.0/24
    ip daddr 10.0.2.0/24
    tcp dport { 8080, 8443 } accept

    # App → DB (TCP 5432)
    ip saddr 10.0.2.0/24
    ip daddr 10.0.3.0/24
    tcp dport 5432 accept

    # Established connections
    ct state established,related accept

    # Log + drop everything else
    log prefix "VM-FW-DROP: " drop
  }
}
```

Libvirt nwfilter (Per-VM)

```
<!-- Simple per-VM filter -->
<filter name='web-server'
  chain='root'>
  <rule action='accept' direction='in'>
    <tcp dstportstart='80' />
  </rule>
  <rule action='accept' direction='in'>
    <tcp dstportstart='443' />
  </rule>
</filter>
```

OVS Flow-Based ACLs

```
# Micro-segmentation with OVS flows
# Allow web-app on port 8080
ovs-ofctl add-flow ovsbr0 \
  "priority=100,ip,\
  nw_src=10.0.1.0/24,\
  nw_dst=10.0.2.0/24,\
  tp_dst=8080,\
  actions=normal"
```

```
</rule>
<rule action='accept' direction='in'>
  <tcp dstportstart='22' />
</rule>
<rule action='drop' direction='inout'>
  <all />
</rule>
</filter>

<!-- Apply to VM interface -->
<interface type='bridge'>
  <filterref filter='web-server' />
</interface>
```

```
# Drop all other inter-tier traffic
ovs-ofctl add-flow ovsbr0 \
  "priority=50,ip,\
  nw_src=10.0.1.0/24,\
  nw_dst=10.0.3.0/24,\
  actions=drop"

# Default: allow same-subnet
ovs-ofctl add-flow ovsbr0 \
  "priority=10,actions=normal"
```

DNS/DHCP & IP Preservation

Zero-downtime network cutover with MAC and IP address preservation.

MAC Address Preservation

```
# h2kvmctl preserves MAC addresses from source VM automatically
h2kvmctl convert --source vm.ova --emit-domain-xml
# Reads OVF: <rasd:Address>00:50:56:ab:12:34</rasd:Address>
# Writes XML: <mac address='00:50:56:ab:12:34' />

# Why MAC preservation matters:
# 1. DHCP reservations continue to work (MAC → IP mapping)
# 2. License servers tied to MAC (e.g., FlexLM, Sentinel)
# 3. Network monitoring tools track by MAC
# 4. ARP tables update automatically – no manual changes

# Override MAC if needed (e.g., VMware MAC prefix conflict)
h2kvmctl convert --source vm.ova --emit-domain-xml --mac-prefix 52:54:00
# Changes 00:50:56:ab:12:34 → 52:54:00:ab:12:34
```

Zero-Downtime Network Cutover Strategy

Pre-Migration (T-1 week)

- Inventory all VM IPs, MACs, VLANs, DNS records
- Document DHCP reservations and static assignments
- Lower DNS TTL to 60 seconds (from default 3600)
- Configure KVM host bridges to match VLAN topology
- Test network connectivity from KVM host to all VLANs
- Verify routing tables and gateway accessibility

During Migration (T-0)

- Shutdown VMware VM (clean shutdown, not force-off)
- Start KVM VM with preserved MAC on same VLAN
- DHCP: lease transfers automatically (same MAC = same IP)
- Static IP: no change needed (same IP on same subnet)
- ARP: gratuitous ARP sent on VM boot (updates switches)
- Typical downtime: 2-5 minutes per VM

DNS Record Updates

- If IP changes: update A/AAAA records
- `nsupdate -k /etc/rndc.key <<EOF`
- `update delete vm1.example.com A`
- `update add vm1.example.com 60 A 10.0.1.50`

Post-Migration Verification

- `ping -c 3 <vm-ip>` — basic connectivity
- `arping -I br0 <vm-ip>` — verify MAC on network
- `traceroute <vm-ip>` — verify routing path
- `nmap -sV <vm-ip>` — verify services responding

- send
- Reverse DNS (PTR): update if IP changed

- Restore DNS TTL to original value after 24 hours
- Monitor for 48 hours before decommissioning VMware VM

Network Validation & Troubleshooting

Post-migration verification checklist and common pitfalls.

Verification Checklist

#	Check	Command	Expected Result	Common Issue
1	VM has network interface	<code>virsh domiflist <vm></code>	Interface on correct bridge	Wrong bridge name in XML
2	Bridge has physical port	<code>bridge link show</code>	Physical NIC in bridge	NIC not added as bridge slave
3	VLAN tagging correct	<code>tcpdump -e -i br0 vlan</code>	Correct VLAN tags on frames	Trunk port not configured on switch
4	MTU matches	<code>ip link show br0</code>	MTU 1500 (or 9000 for jumbo)	MTU mismatch causes fragmentation
5	VM can reach gateway	<code>ping -c 3 <gateway></code>	0% packet loss	IP forwarding disabled on host
6	DNS resolution works	<code>nslookup example.com</code>	Valid response	resolv.conf not configured in guest
7	TCP throughput	<code>iperf3 -c <target> -t 30</code>	>8 Gbps on 10GbE	Checksum offload disabled
8	No packet loss under load	<code>iperf3 -c <target> -u -b 10G</code>	<0.1% loss	Ring buffer too small

Common Pitfalls & Fixes

MTU Mismatch (Most Common)

- **Symptom:** SSH works but large transfers stall
- **Cause:** VMware used jumbo frames (MTU 9000), KVM bridge defaults to 1500
- **Fix:** Match MTU across path: `ip link set br0 mtu 9000`
- Also set on physical NIC: `ip link set ens192 mtu 9000`
- And in guest: `ip link set eth0 mtu 9000`
- Verify: `ping -M do -s 8972 <target>`

IP Forwarding Disabled

- **Symptom:** VMs can't reach outside subnet
- **Cause:** Linux kernel drops forwarded packets by default
- **Fix:** `sysctl -w net.ipv4.ip_forward=1`
- Persistent: `echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-kvm.conf`
- For IPv6: `net.ipv6.conf.all.forwarding=1`
- Also check nftables isn't blocking FORWARD chain

Checksum Offload Issues

- **Symptom:** tcpdump shows bad checksums, intermittent drops
- **Cause:** Hardware offload mismatch between host and guest
- **Fix in guest:** `ethtool -K eth0 tx off rx off`
- Better fix: ensure virtio NIC has csum enabled
- libvirt XML: `<driver name='vhost' csum='on' gso='on'/>`
- Verify: `ethtool -k eth0 | grep checksum`

Bridge Filtering (iptables/nftables)

- **Symptom:** Bridged traffic silently dropped
- **Cause:** bridge-nf-call-iptables sends L2 traffic through L3 filter
- **Fix:** Disable bridge filtering:
 - `sysctl -w net.bridge.bridge-nf-call-iptables=0`
 - `sysctl -w net.bridge.bridge-nf-call-ip6tables=0`
 - Or add explicit ACCEPT rules in FORWARD chain

Network Migration Success: Match Topology, Preserve MACs, Verify Path

90% of post-migration network issues are caused by three things: MTU mismatch, missing IP forwarding, or wrong bridge assignment. h2kvmctl auto-maps VMware network topology to Linux equivalents, preserves MAC addresses, and generates ready-to-use libvirt network XML. Run the 8-point verification checklist after each VM migration.